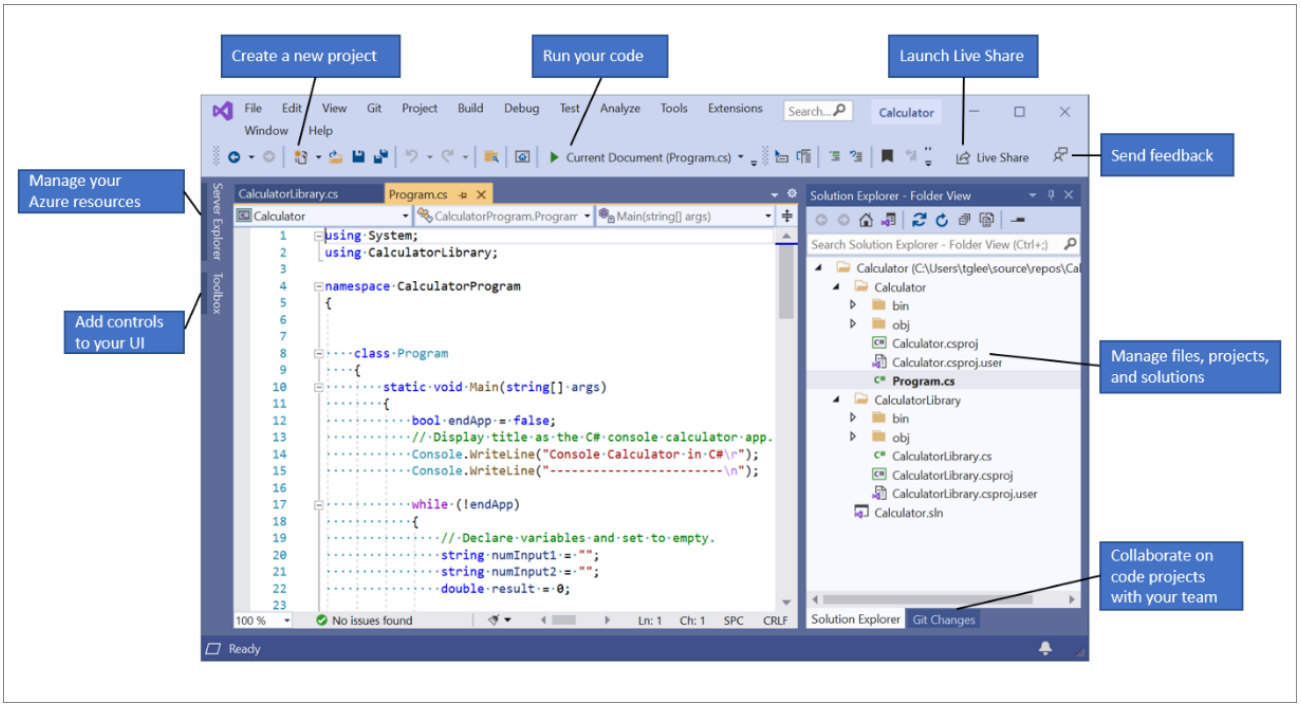
C# Windows Form

C# is a general-purpose, modern and object-oriented programming language pronounced as **"C sharp"**. C# is a lot similar to Java syntactically and is easy for the users who have knowledge of C, C++ or Java.

C# Programming language will allow developers to build a variety of secure and robust applications such as windows applications, web applications, database applications, etc., which will run on the **.NET Framework**.

The .NET Framework is a software development framework that provides a runtime environment (CLR) and a set of libraries and tools for building and running applications on Windows operating systems.

Microsoft provided an IDE (Integrated Development Environment) tool called Visual Studio to implement C# programs easily. The figure below is the visual studio interface for a C # console application.



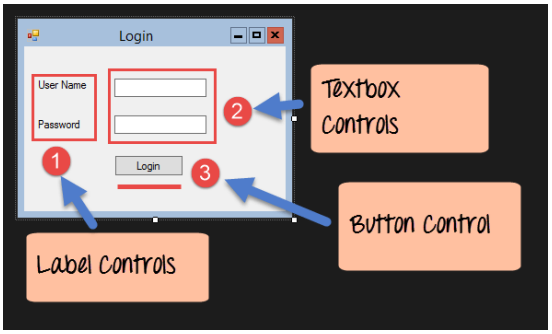
Why C#? C# has many reasons for being popular and in demand. Few of the reasons are mentioned below:

1. **Easy to start:** C# is a high-level language so it is closer to other popular programming languages like C, C++, and Java and thus becomes easy to learn for anyone.

2. C# provides **automatic memory management** by clearing unused objects.
3. **Widely used for developing Desktop and Web Application:** C# is widely used for developing web applications and Desktop applications. It is one of the most popular languages that is used in professional desktop. If anyone wants to create Microsoft apps, C# is their first choice.
4. **Large standard library:** C# has a large standard library that includes a wide range of pre-built classes and functions. This makes it easy for developers to perform common tasks without having to write a lot of custom code.
5. **Object-oriented programming:** C# is a fully object-oriented language, which allows developers to create reusable code and build complex applications with ease.
6. **Community:**The larger the community the better it is as new tools and software will be developing to make it better. C# has a large community so the developments are done to make it exist in the system and not become extinct.
7. **Game Development:** C# is widely used in game development and will continue to dominate. C# integrates with Microsoft and thus has a large target audience.

Windows Forms

Windows Forms is a Graphical User Interface (GUI) class library which is bundled in *.Net Framework*. Its main purpose is to provide an easier interface to develop the applications for desktop, tablet, PCs. It is also termed as the **WinForms**. The applications which are developed by using WinForms are known as the **Windows Forms Applications** that runs on the desktop computer. WinForms applications can contain the different type of controls like labels, text boxes, buttons, etc., as shown in the figure below.

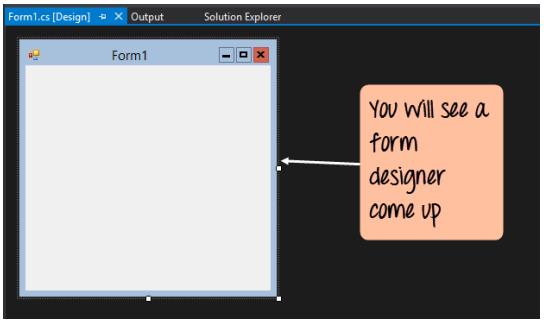


Windows Forms applications are versatile and can be used to create various types of applications such as data entry, management, and reporting applications, as well as games and multimedia applications.

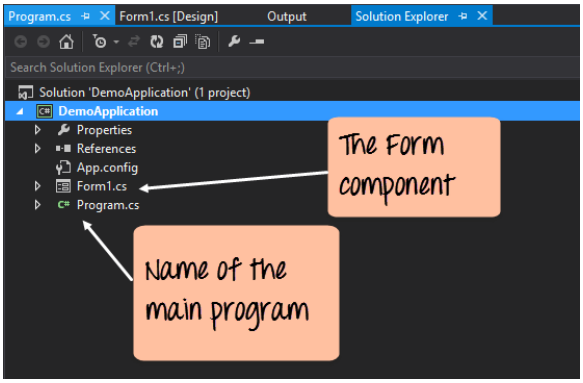
- First, open the Visual Studio then Go to **File -> New -> Project** to create a new project and then select the language as *Visual C#* from the left menu. Click on ***Windows Forms App (.NET Framework)*** in the middle of current window.

- After that following window will display which will be divided into three parts as follows:

1. **Editor Window or Main Window:** Here, you will work with forms and code editing. You can notice the layout of form which is now blank. You will double click the form then it will open the code for that.



2. **Solution Explorer Window:** It is used to navigate between all items in solution. For example, if you will select a file form this window then particular information will be display in the property window.

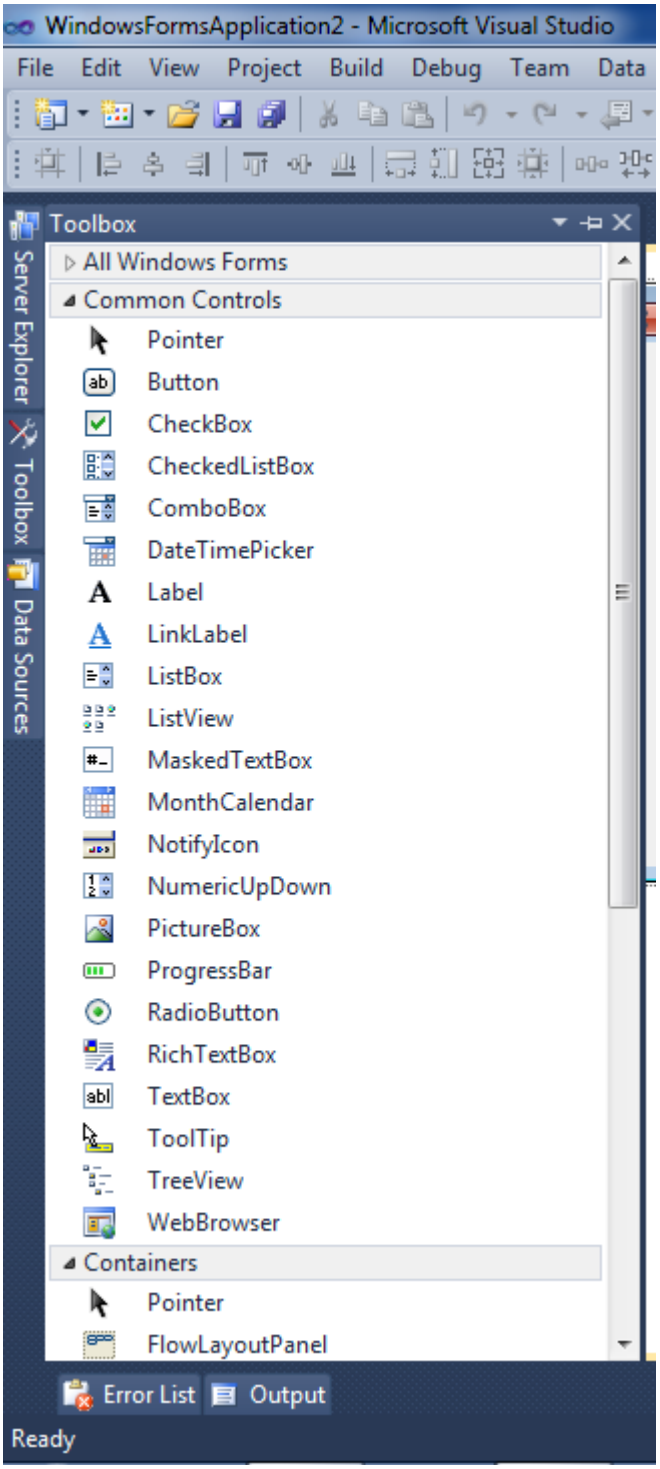


- A Form application called **Forms1.cs.** This file will contain all of the code for the Windows Form application.

- The Main program called **Program.cs** is default code file which is created when a new application is created in Visual Studio. This code will contain the startup code for the application as a whole.

3. **Properties Window:** This window is used to change the different properties of the selected item in the Solution Explorer. Also, you can change the properties of components or controls that you will add to the forms.

Common Controls



Textbox: Allows the user to select, edit, and copy text. It is an interactive input field.

Label: Displays text but does not allow the user to select, edit, or copy it (unless specifically enabled through code or properties).

ComboBox: A drop-down list that allows users to select **one** item from a predefined list.

ListBox: Displays a list of items, allowing users to select one or multiple items.

DataGridView (Grid Control): Used to **display tabular data** in rows and columns.

PictureBox (Image Control): Used to display images like .jpg, .png, .bmp, etc.

Let's establish a naming convention for the controls.

Textbox:	**Txt**
List:	**Lst**
Button:	**Btn**
Label:	**Lbl**
Combo box:	**Cbo**
Image:	**Img**
Grid:	**Grd**

We will use the PascalCase naming convention where the first letter of each compound word in a variable is capitalized.

Event Handlers (Windows Forms)

An event handler is a method that is bound to an event. When the event is raised, the code within the event handler is executed. Each event handler provides two parameters that allow you to handle the event properly. The following example shows an event handler for a Button control's Click event.

```
private void button1_Click(object sender, System.EventArgs e)
{
}
```

The first parameter, **sender**, provides a reference to the object that raised the event.

The second parameter, **e**, in the example above, passes an object specific to the event that is being handled.

You can also use the same event handler to handle the same event for different controls.

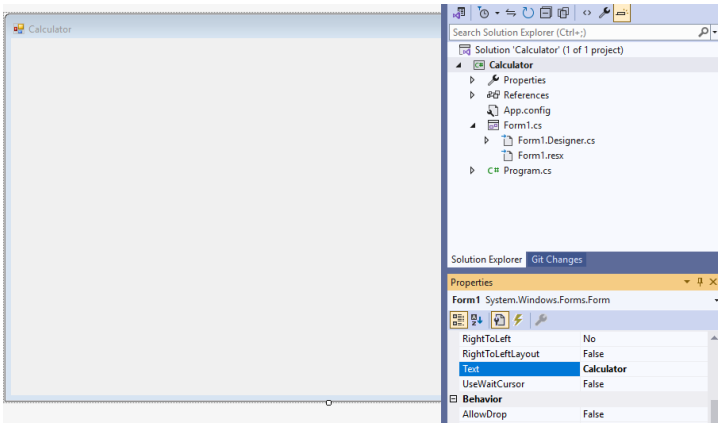
Create your first program: Calculator Program

This app is capable of performing addition, subtraction, multiplication as well as division. So let's get started.

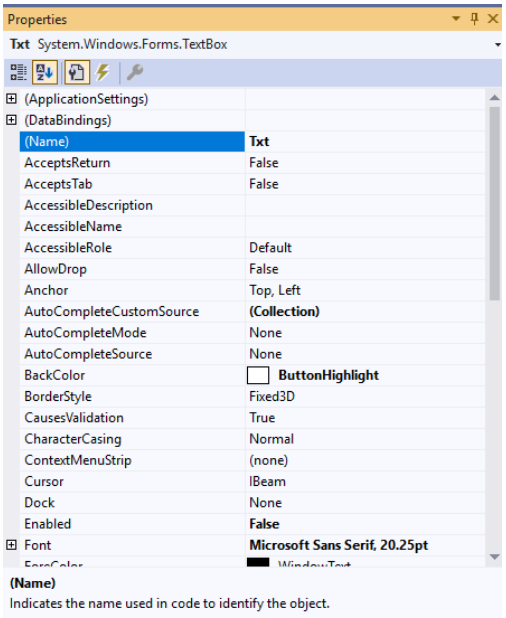
1. Create a New Project based on **WinForm**.

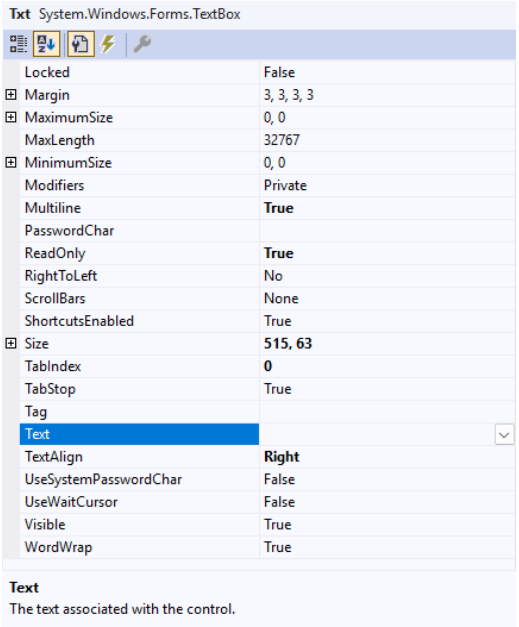
2. Set its type to **Windows Forms Application** and set its name to **CalculatorApp**. Press OK.

3. Then change the Text form property to **Calculator**, because we don't want our application to have the title **Form1** when it starts.



4. Add a **TextBox** control to display the calculation results. Set its **ReadOnly** property to **True**, change the **background color to white**, **adjust the font size**, and **set the text alignment**.



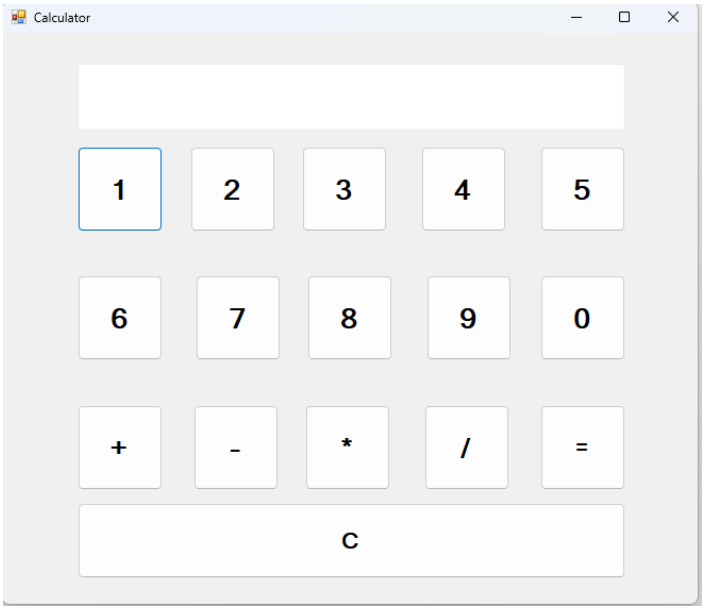


5. Now add the calculator buttons: numbers (0-9), operands (+,-,*, /) , equal (result) and the clear button.

6. From the tool box window, drag and drop a **Button** onto the form.

From the Properties tab, change its **Name** property to **BtnN1** for button number 1 , **Text** property **into 1.**

You can set the font type /size, the background color and the button size. Repeat that for the other numbers.



7. Double-click the button named **BtnN1** to navigate to its event handler. The procedure **BtnN1_Click** is executed when the user clicks the button **BtnN1**.

First initialize the following variables in order to perofrm the calculations:

```
namespace Calculator
{
    3 references
    public partial class CalculatorApp : Form
    {

        1 reference
        public CalculatorApp()
        {
            InitializeComponent();
        }

        double FirstNumber;
        double SecondNumber;
        string operation;
        double result;
```

Now, add the following code in **BtnN1_Click** procedure to display the number **1** in the textbox (Txt).

```
1 reference
private void BtnN1_Click(object sender, EventArgs e)
{
    Txt.Text = Txt.Text + "1";


}
```

Repeat for all the number buttons (0,2-9)

8. Next, create the clear button, and the operands buttons. For the clear button set the **name to BtnC,** and the **text property to C**. Similarly create add, sub, mul and div buttons.

9. Now for the **Addition button** add the following code:

```
1 reference
private void BtnAdd_Click(object sender, EventArgs e)
{
    operation = "+";
    FirstNumber=double.Parse(Txt.Text);

    Txt.Clear();

}
```

Convert String to Double for Calculations

If you receive numeric data in string format, you cannot perform arithmetic operations on it. Double.Parse method allows conversion to a usable numeric format. The Double.Parse method belongs to the **System** namespace in **C#**.

So it converts the string representation of a number to its double-precision floating-point number equivalent.

Task: Complete the program.

References

https://www.geeksforgeeks.org/introduction-to-c-sharp-windows-forms-applications/
https://www.geeksforgeeks.org/introduction-to-c-sharp/?ref=ml_lbp
https://www.guru99.com/c-sharp-windows-forms-application.html
https://www.geeksforgeeks.org/csharp-programming-language/
https://www.c-sharpcorner.com/article/create-basic-calculator-using-windows-forms-and-c-sharp/
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/event-handlers-overview-windows-forms?view=netframeworkdesktop-4.8